

IssueFlow

Plan del proyecto, diseño de dominio y apéndice en Markdown

Propósito	Definir una versión realizable de un sistema de gestión de tickets/incidencias basado en microser
Stack objetivo	Gateway en Node.js, Ticket Service en Spring Boot, Assignment Service en FastAPI, Notification
Ruta técnica	Construcción local -> Docker Compose -> despliegue en EC2 -> migración a ECS/Fargate -> mig

Este documento consolida el plan funcional y técnico de **IssueFlow**, incluyendo la visión de negocio, arquitectura de microservicios, fases de implementación, entidades de dominio y un apéndice con el mismo contenido en formato Markdown para copiar y pegar en VS Code.

1. Visión de negocio

IssueFlow es una plataforma para registrar, asignar, atender y dar seguimiento a tickets e incidencias. Está pensado para pequeñas organizaciones que hoy gestionan solicitudes por WhatsApp, correo, llamadas o hojas de cálculo y necesitan trazabilidad operativa.

Problemas que resuelve:

- falta de trazabilidad y pérdida de casos
- desorden en la asignación de responsables
- ausencia de historial auditable
- dificultad para medir tiempos de atención y estados del flujo

Roles principales:

- Solicitante: crea tickets.
- Agente: atiende tickets.
- Administrador o coordinador: asigna y supervisa tickets.

2. Alcance funcional V1

- crear ticket
- listar tickets
- consultar detalle de ticket
- asignar ticket a un agente
- cambiar estado del ticket
- registrar eventos de auditoría

Fuera de V1:

- autenticación compleja
- dashboard avanzado
- archivos adjuntos
- SLA y reglas complejas
- observabilidad avanzada
- CI/CD completo desde el día 1

3. Microservicios propuestos

Gateway Service - Node.js

Punto único de entrada. Recibe peticiones del cliente, enruta hacia los otros servicios y más adelante puede centralizar autenticación y composición de respuestas.

Ticket Service - Spring Boot

Corazón del dominio. Crea tickets, consulta tickets, cambia estados y coordina asignación y auditoría. Aquí conviene aplicar arquitectura hexagonal.

Assignment Service - FastAPI

Gestiona asignaciones. Valida agentes, asigna tickets y mantiene lógica básica de reparto.

Notification Audit Service - Rails API o Sinatra

Registra eventos y notificaciones simples. Mantiene el historial auditable del sistema.

4. Flujo de negocio principal

- El solicitante crea un ticket.
- El ticket inicia en estado OPEN.
- El coordinador o una regla solicita asignación.
- Assignment Service devuelve el agente asignado.
- Ticket Service actualiza el ticket a ASSIGNED.
- El agente inicia atención y el ticket pasa a IN_PROGRESS.
- El agente resuelve el ticket y pasa a RESOLVED.
- Finalmente el ticket puede pasar a CLOSED.
- Cada cambio relevante se registra en auditoría.

Estados recomendados:

Estado	Descripción breve
OPEN	Ticket recién creado, aún sin asignar.
ASSIGNED	Ya tiene un agente responsable.
IN_PROGRESS	Se encuentra en atención activa.
RESOLVED	La solución fue aplicada.
CLOSED	Caso cerrado formalmente.

5. Comunicación entre servicios

Versión inicial recomendada: todo por HTTP para reducir complejidad. La evolución futura puede mover auditoría y notificaciones a un flujo asíncrono con RabbitMQ o SQS.

- Gateway -> Ticket Service: HTTP
- Ticket Service -> Assignment Service: HTTP
- Ticket Service -> Notification Audit Service: HTTP

6. Fases del proyecto

Fase 0 - Definición y alcance

Definir la visión de negocio, responsabilidades por servicio, flujo principal y límites de la V1.

Fase 1 - Diseño funcional y técnico

Definir entidades, estados, endpoints, contratos entre servicios, bases de datos y estructura de carpetas.

Fase 2 - Construcción local

Levantar los cuatro servicios localmente, implementar health checks, endpoints mínimos y persistencia básica.

Fase 3 - Dockerización

Crear un Dockerfile por servicio, configurar variables de entorno y orquestar con Docker Compose.

Fase 4 - Despliegue en EC2

Levantar toda la solución en una sola VM con Docker Compose para aprender el modelo convencional.

Fase 5 - Despliegue en ECS

Subir imágenes a ECR y desplegar como servicios administrados, idealmente con Fargate.

Fase 6 - Despliegue en EKS

Reutilizar imágenes en ECR y desplegar con Deployments, Services e Ingress o LoadBalancer.

Fase 7 - Evolución posterior

Agregar mensajería asíncrona, reglas automáticas, observabilidad, autenticación y CI/CD.

7. Entidades de dominio

Ticket Service (dominio principal)

Ticket: representa una incidencia o solicitud reportada en el sistema.

- Atributos sugeridos: id, title, description, status, priority, category, requesterId, assigneeId, createdAt, updatedAt, resolvedAt, closedAt.
- Reglas: inicia en OPEN; no puede pasar de OPEN a CLOSED directamente; no se puede cerrar si no está RESOLVED; no se puede asignar sin assigneeId.

TicketComment: opcional para V1.1; sirve para observaciones operativas.

Assignment Service

Agent: persona o recurso que puede recibir tickets.

Assignment: relación entre ticket y agente, con historial posible de asignaciones.

Notification Audit Service

AuditEvent: evento importante del sistema con eventType, ticketId, performedBy, payload y occurredAt.

NotificationLog: opcional en V1, útil si se quiere diferenciar auditoría de notificación simulada.

Objetos de valor / enums

- TicketStatus: OPEN, ASSIGNED, IN_PROGRESS, RESOLVED, CLOSED.
- TicketPriority: LOW, MEDIUM, HIGH, CRITICAL.
- TicketCategory: SOFTWARE, HARDWARE, ACCESS, NETWORK, OTHER.
- AssignmentStatus: ACTIVE, REASSIGNED, REMOVED.
- AuditEventType: TICKET_CREATED, TICKET_ASSIGNED, STATUS_CHANGED, TICKET_RESOLVED, TICKET_CLOSED.

8. Casos de uso principales

Ticket Service

- CreateTicket
- GetTicketById
- ListTickets
- UpdateTicketStatus
- AssignTicketToAgent

Assignment Service

- AssignAgentToTicket
- GetAgentById
- ListAvailableAgents

Notification Audit Service

- RegisterAuditEvent
- ListAuditEventsByTicket

9. Estructura base sugerida

Se recomienda aplicar arquitectura hexagonal principalmente en Ticket Service. Los otros servicios deben mantenerse simples y bien delimitados para no sobrecargar la primera iteración.

Apéndice A. Contenido equivalente en Markdown

A continuación se incluye una versión Markdown del plan para copiar y pegar en VS Code. El bloque usa formato monoespaciado dentro del PDF para preservar la estructura.

```
# IssueFlow - Plan consolidado

## 1. Vision de negocio
IssueFlow es una plataforma para registrar, asignar, atender y dar seguimiento a tickets e incidencias.

### Problemas que resuelve
- falta de trazabilidad
- tickets perdidos
- desorden en la asignacion de responsables
- ausencia de historial auditable

### Roles principales
- Solicitante
- Agente
- Administrador / Coordinador

## 2. Alcance funcional V1
Incluye:
- crear ticket
- listar tickets
- consultar detalle
- asignar ticket
- cambiar estado
- registrar auditoria

No incluye en V1:
- autenticacion compleja
- dashboard avanzado
- archivos adjuntos
- SLA complejos
- observabilidad avanzada
- CI/CD completo desde el dia 1

## 3. Microservicios
### Gateway Service - Node.js
Punto unico de entrada. Recibe peticiones y enruta.

### Ticket Service - Spring Boot
Corazon del dominio. Aqui conviene aplicar arquitectura hexagonal.

### Assignment Service - FastAPI
Gestiona asignaciones, agentes y validaciones basicas.

### Notification Audit Service - Rails API o Sinatra
Registra eventos y notificaciones simples.

## 4. Flujo de negocio
1. El solicitante crea un ticket.
2. El ticket inicia en OPEN.
3. Se solicita asignacion.
4. Assignment Service devuelve agente.
5. Ticket Service actualiza a ASSIGNED.
6. El agente pasa a IN_PROGRESS.
7. Luego pasa a RESOLVED.
8. Finalmente puede pasar a CLOSED.
```

```
9. Cada cambio relevante se registra en auditoria.

## 5. Estados recomendados
- OPEN
- ASSIGNED
- IN_PROGRESS
- RESOLVED
- CLOSED

## 6. Comunicacion entre servicios
- Gateway -> Ticket Service: HTTP
- Ticket Service -> Assignment Service: HTTP
- Ticket Service -> Notification Audit Service: HTTP

## 7. Fases del proyecto
### Fase 0 - Definicion y alcance
Definir vision, flujo y limites de la V1.

### Fase 1 - Diseno funcional y tecnico
Definir entidades, estados, endpoints y contratos.

### Fase 2 - Construccion local
Levantar los cuatro servicios localmente.

### Fase 3 - Dockerizacion
Dockerfile por servicio y Docker Compose.

### Fase 4 - EC2
Desplegar todo en una sola VM.

### Fase 5 - ECS
Subir imagenes a ECR y desplegar servicios.

### Fase 6 - EKS
Desplegar con Deployments, Services e Ingress.

### Fase 7 - Evolucion posterior
Agregar mensajeria, observabilidad, autenticacion y CI/CD.

## 8. Entidades de dominio
### Ticket Service
- Ticket
- TicketComment (opcional)

### Assignment Service
- Agent
- Assignment

### Notification Audit Service
- AuditEvent
- NotificationLog (opcional)

## 9. Objetos de valor
### TicketStatus
```text
```



```
OPEN
ASSIGNED
IN_PROGRESS
RESOLVED
CLOSED
```

Transiciones recomendadas:
- OPEN -> ASSIGNED
- ASSIGNED -> IN_PROGRESS
- IN_PROGRESS -> RESOLVED
- RESOLVED -> CLOSED

### TicketPriority
```text
LOW
MEDIUM
HIGH
CRITICAL
```

### TicketCategory
```text
SOFTWARE
HARDWARE
ACCESS
NETWORK
OTHER
```

### AssignmentStatus
```text
ACTIVE
REASSIGNED
REMOVED
```

### AuditEventType
```text
TICKET_CREATED
TICKET_ASSIGNED
STATUS_CHANGED
TICKET_RESOLVED
TICKET_CLOSED
```

## 10. Casos de uso principales
### Ticket Service
- CreateTicket
- GetTicketById
- ListTickets
- UpdateTicketStatus
- AssignTicketToAgent

### Assignment Service
```

```
- AssignAgentToTicket
- GetAgentById
- ListAvailableAgents

### Notification Audit Service
- RegisterAuditEvent
- ListAuditEventsByTicket

## 11. Estructura sugerida
### Ticket Service (Spring Boot, hexagonal)
```text
ticket-service/
 src/main/java/com/issueflow/ticket/
 domain/
 model/
 Ticket.java
 TicketStatus.java
 TicketPriority.java
 TicketCategory.java
 port/
 in/
 CreateTicketUseCase.java
 GetTicketUseCase.java
 ListTicketsUseCase.java
 UpdateTicketStatusUseCase.java
 AssignTicketUseCase.java
 out/
 TicketRepository.java
 AssignmentClient.java
 AuditClient.java
 application/
 service/
 CreateTicketService.java
 GetTicketService.java
 ListTicketsService.java
 UpdateTicketStatusService.java
 AssignTicketService.java
 infrastructure/
 persistence/
 web/
 client/
 ...

Assignment Service (FastAPI)
```text
assignment-service/
  app/
    domain/
      models/
        agent.py
        assignment.py
      enums/
        assignment_status.py
    services/
      assignment_service.py
```

```
repositories/  
  agent_repository.py  
  assignment_repository.py  
api/  
  routes/  
    assignments.py  
    agents.py  
...  
  
### Notification Audit Service (Rails API o Sinatra)  
``text  
notification-audit-service/  
  app/  
    models/  
      audit_event.rb  
    controllers/  
      audit_events_controller.rb  
    services/  
      audit_event_service.rb  
...
```

Apéndice B. PlantUML - Diagrama de componentes

```
@startuml
title IssueFlow - Diagrama de Componentes

package "Client Layer" {
    [Web Client / API Consumer] as Client
}

package "Gateway Service (Node.js)" {
    [Gateway Controller]
    [Routing / Aggregation Component]
}

package "Ticket Service (Spring Boot)" {
    [Ticket REST Controller]
    [Ticket Application Services]
    [Ticket Domain Model]
    [Ticket Repository Port]
    [Assignment Client Port]
    [Audit Client Port]
    [JPA Ticket Repository Adapter]
    [HTTP Assignment Client Adapter]
    [HTTP Audit Client Adapter]
}

package "Assignment Service (FastAPI)" {
    [Assignment API]
    [Assignment Service]
    [Agent Repository]
    [Assignment Repository]
}

package "Notification Audit Service (Rails/Sinatra)" {
    [Audit API]
    [Audit Event Service]
    [Audit Repository]
}

database "Ticket DB" as TicketDB
database "Assignment DB" as AssignmentDB
database "Audit DB" as AuditDB

Client --> [Gateway Controller]
[Gateway Controller] --> [Routing / Aggregation Component]
[Routing / Aggregation Component] --> [Ticket REST Controller]
[Ticket REST Controller] --> [Ticket Application Services]
[Ticket Application Services] --> [Ticket Domain Model]
[Ticket Application Services] --> [Ticket Repository Port]
[Ticket Application Services] --> [Assignment Client Port]
[Ticket Application Services] --> [Audit Client Port]
[Ticket Repository Port] <.. [JPA Ticket Repository Adapter]
[Assignment Client Port] <.. [HTTP Assignment Client Adapter]
[Audit Client Port] <.. [HTTP Audit Client Adapter]
[JPA Ticket Repository Adapter] --> TicketDB
[HTTP Assignment Client Adapter] --> [Assignment API]
[HTTP Audit Client Adapter] --> [Audit API]
[Assignment API] --> [Assignment Service]
[Assignment Service] --> [Agent Repository]
[Assignment Service] --> [Assignment Repository]
[Agent Repository] --> AssignmentDB
[Assignment Repository] --> AssignmentDB
[Audit API] --> [Audit Event Service]
[Audit Event Service] --> [Audit Repository]
[Audit Repository] --> AuditDB
@enduml
```